

Predicting Online Shopper Purchase Intention

A Comparative Study of KNN, Logistic Regression, and Decision Tree Classifiers

Machine Learning Project Documentation

Dataset: Online Shoppers Purchasing Intention (UCI Machine Learning Repository)

1. Introduction

This project investigates whether a visitor's browsing behaviour on an e-commerce website can be used to predict whether that session will end in a purchase. The target variable is Revenue, a boolean flag indicating whether the session resulted in a transaction.

Three supervised classification algorithms were implemented and compared on the same dataset: K-Nearest Neighbors (KNN), Logistic Regression, and a Decision Tree Classifier. Each model follows the same overall pipeline — data cleaning, encoding, train/test splitting, feature selection, and (where required) feature scaling — before being trained and evaluated on an identical 80/20 train-test split (`random_state = 42`).

Dataset source: Online Shoppers Purchasing Intention Dataset, UCI Machine Learning Repository. It contains 12,330 website sessions collected over a one-year period, each described by 17 behavioural and technical features plus the Revenue target.

2. Dataset Overview

The table below summarises the raw dataset before any cleaning was applied.

Property	Value
Number of rows (sessions)	12,330
Number of columns	18 (17 features + Revenue target)
Missing values	0
Duplicate rows	125
Target variable	Revenue (True / False)
Class balance	10,422 No Purchase (84.5%) / 1,908 Purchase (15.5%)

Feature columns:

Administrative, Administrative_Duration, Informational, Informational_Duration, ProductRelated, ProductRelated_Duration, BounceRates, ExitRates, PageValues, SpecialDay, Month, OperatingSystems, Browser, Region, TrafficType, VisitorType, Weekend.

2.1 Target Distribution

The dataset is noticeably imbalanced — only about 1 in 6 sessions results in a purchase. This imbalance was addressed differently across the three models (see Section 3.2, 4.2 and 5.2).

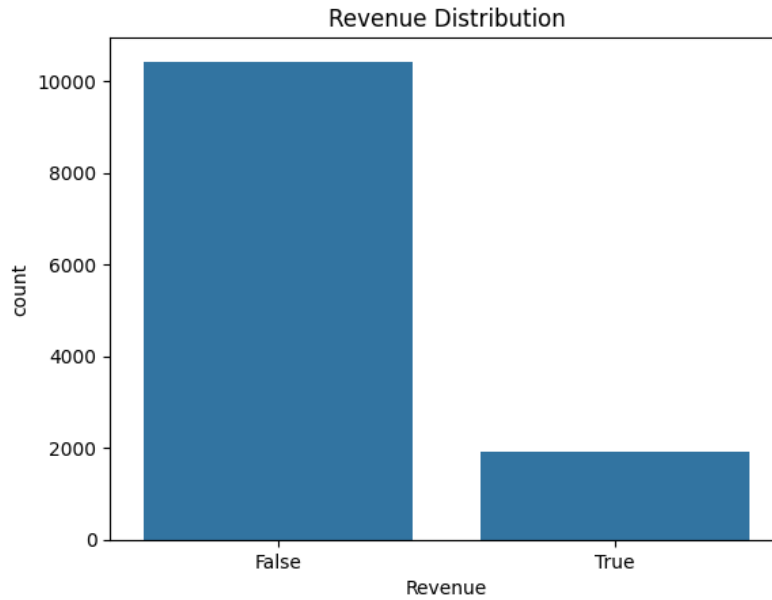


Figure 1 — Distribution of the Revenue target variable

2.2 Data Cleaning

- 125 duplicate rows were identified and removed.
- No missing values were present in the dataset, so no imputation was required.
- Categorical columns (Revenue, Weekend, VisitorType, Month) were label-encoded into numeric form.

3. Model 1 — K-Nearest Neighbors (KNN)

3.1 Feature Selection

SelectKBest with `mutual_info_classif` was used to select the 10 most informative features, fitted only on the training data to avoid data leakage into the test set.

Feature	Selected
Administrative_Duration	✓
Informational_Duration	✓
ProductRelated	✓
ProductRelated_Duration	✓
BounceRates	✓
ExitRates	✓
PageValues	✓
SpecialDay	✓
TrafficType	✓
VisitorType	✓

3.2 Handling Class Imbalance

SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training set only, generating synthetic examples of the minority class (purchasers) so the model does not simply learn to predict the majority class.

3.3 Choosing the Best K

Odd and even values of K from 3 up to $\sqrt{\text{(training size)}}$ were tested, scoring each by the average of accuracy and F1 score. The best-performing value was K = 18 (distance-weighted voting).

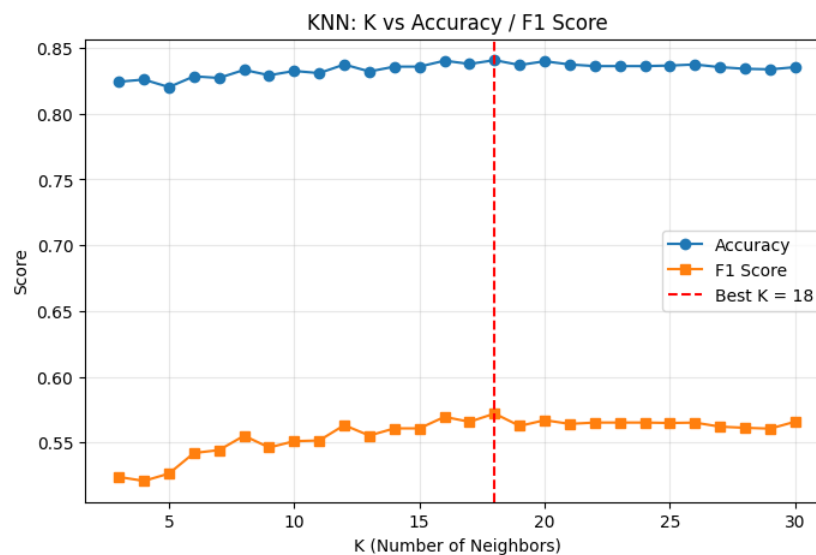


Figure 2 — Accuracy and F1 score across different values of K

3.4 Evaluation

Metric	Value
Accuracy	0.8406

Precision (Class 1 — Purchase)	0.4753
Recall (Class 1 — Purchase)	0.7182
F1 Score (Class 1)	0.5721

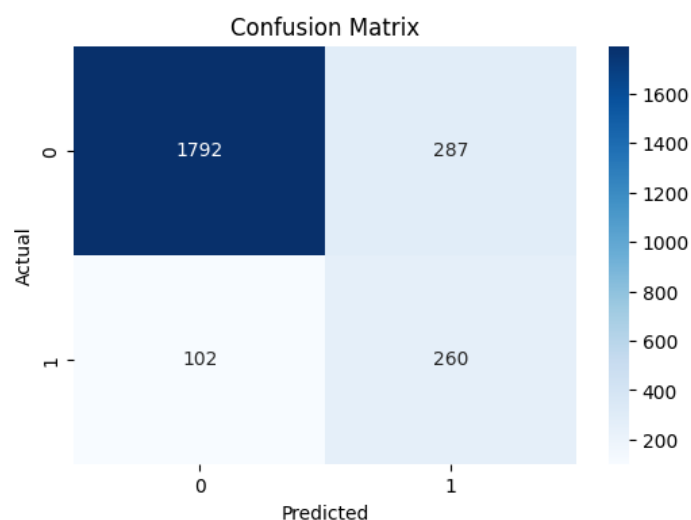


Figure 3 — KNN Confusion Matrix

Interpretation: The model correctly identifies about 72% of actual purchasers (recall), which is valuable for a business that wants to catch as many potential buyers as possible for retargeting — at the cost of a moderate number of false positives.

4. Model 2 — Logistic Regression

4.1 Feature Selection

The same SelectKBest / mutual_info_classif approach was used, fitted on the training set after the train-test split.

Feature	Selected
Administrative	✓
Administrative_Duration	✓
ProductRelated	✓
ProductRelated_Duration	✓
BounceRates	✓
ExitRates	✓
PageValues	✓
Month	✓
TrafficType	✓
VisitorType	✓

4.2 Handling Class Imbalance

class_weight='balanced' was passed to LogisticRegression, which automatically re-weights the loss function in inverse proportion to class frequency — penalising misclassification of the minority (purchase) class more heavily.

4.3 Evaluation

Metric	Value
Accuracy	0.8726
Precision (Class 1 — Purchase)	0.5539
Recall (Class 1 — Purchase)	0.7238
F1 Score (Class 1)	0.6275

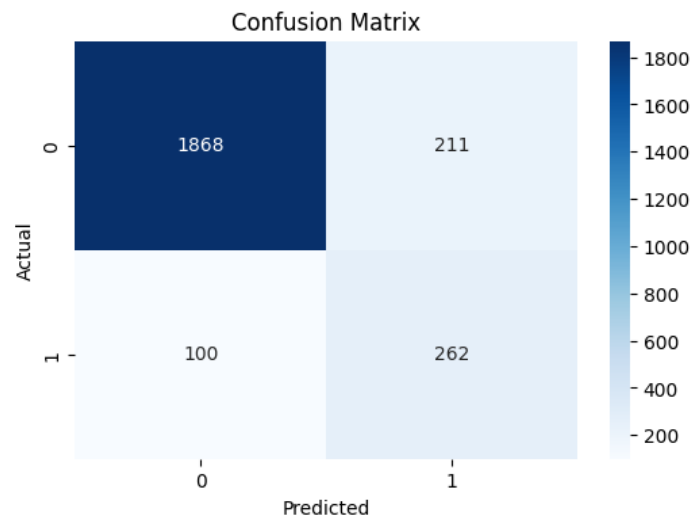


Figure 4 — Logistic Regression Confusion Matrix

4.4 Feature Coefficients

Coefficients indicate each feature's direction and strength of influence on the log-odds of a purchase (positive = increases purchase likelihood).

Feature	Coefficient
PageValues	+2.148
Month	+0.240
ProductRelated	+0.203
ProductRelated_Duration	+0.184
Administrative	+0.042
Administrative_Duration	+0.022
TrafficType	+0.019
VisitorType	-0.146
BounceRates	-0.193
ExitRates	-0.482

PageValues is by far the strongest positive predictor — sessions with higher page values (a Google Analytics metric tied to proximity to conversion) are much more likely to end in a purchase, which matches business intuition.

5. Model 3 — Decision Tree

5.1 Feature Selection

Same SelectKBest approach, fitted on training data only. The selected features were identical to those chosen for Logistic Regression.

5.2 Handling Class Imbalance

`class_weight='balanced'` was used, and the tree depth was constrained (`max_depth = 3`) to prevent overfitting on the noisy, imbalanced data.

5.3 Evaluation

Metric	Value
Accuracy	0.8533
Precision (Class 1 — Purchase)	0.5034
Recall (Class 1 — Purchase)	0.8177
F1 Score (Class 1)	0.6232

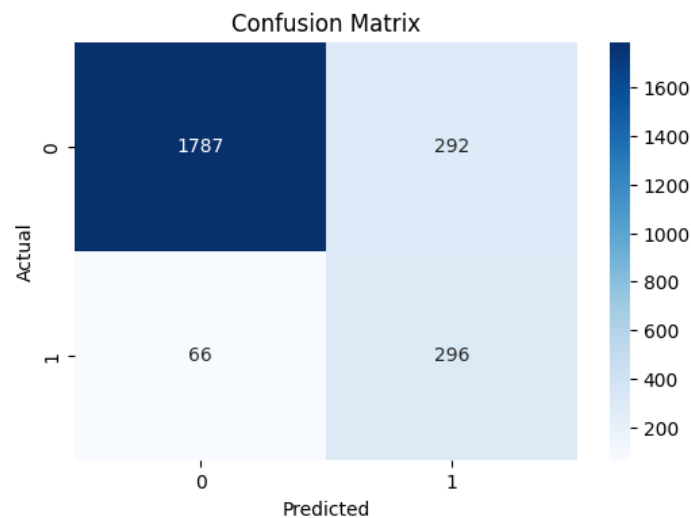


Figure 5 — Decision Tree Confusion Matrix

5.4 Feature Importance

Feature	Importance
PageValues	0.8636
Month	0.0998
ProductRelated	0.0184
BounceRates	0.0149
Administrative	0.0034

As with Logistic Regression, PageValues dominates the model's decisions — it alone accounts for roughly 86% of the tree's total information gain.

5.5 Tree Visualization

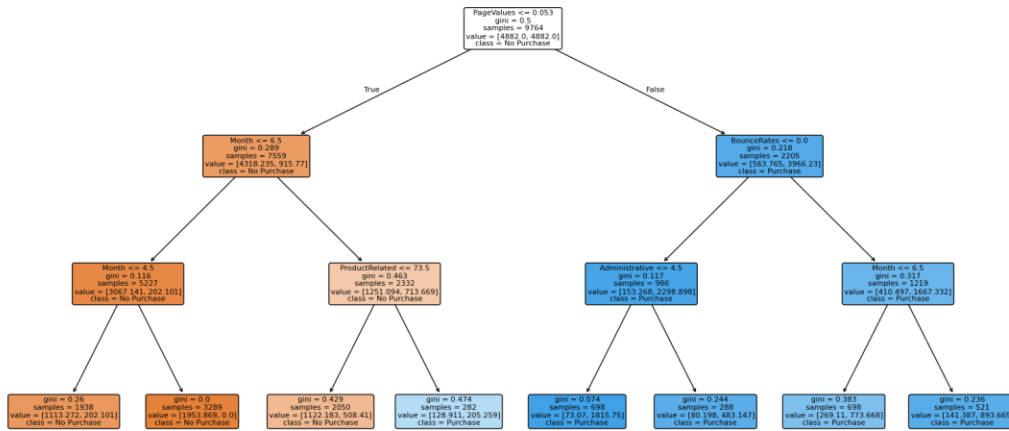


Figure 6 — Trained Decision Tree (max depth = 3)

6. Model Comparison

All three models were trained and evaluated on the identical 80/20 split of the same 10 selected features, making the comparison below fair and consistent.

Model	Accuracy	Precision	Recall	F1 Score
K-Nearest Neighbors	0.8406	0.4753	0.7182	0.5721
Logistic Regression	0.8726	0.5539	0.7238	0.6275
Decision Tree	0.8533	0.5034	0.8177	0.6232

6.1 Discussion

- Logistic Regression achieved the highest accuracy (87.3%) and precision (0.554), making it the most reliable when a false 'will purchase' alarm is costly.
- Decision Tree achieved the highest recall (0.818), catching the largest share of actual purchasers — useful for marketing campaigns where missing a genuine buyer is more costly than a false positive.
- KNN performed reasonably but was outperformed by both other models on every metric; it is also the most computationally expensive at prediction time since it stores the full training set.
- All three models share PageValues, ProductRelated(_Duration), Month, and ExitRates/BounceRates as their most influential features, giving confidence that these signals are genuinely predictive rather than model-specific artefacts.

7. Conclusion

This project demonstrated a complete, leakage-free machine learning pipeline — cleaning, encoding, feature selection, imbalance handling, training, and evaluation — applied consistently across three different classification algorithms on the same online shopper dataset.

For a business use case such as targeted retargeting of likely buyers, the Decision Tree's high recall (82%) is attractive, since missing a genuine buyer carries a real opportunity cost. For a use case where false positives are expensive (e.g. triggering an expensive personalised discount), Logistic Regression's higher precision is preferable.

Limitations

- The dataset is significantly imbalanced (84.5% / 15.5%); while SMOTE and class_weight mitigate this, results on the minority class remain less reliable than on the majority class.
- Only 10 of 17 available features were used per model; a broader hyperparameter and feature-count search may further improve results.
- No cross-validation was used — a single 80/20 split was evaluated, so reported metrics carry some sampling variance.